

D-42 record

D-42 — failed classifier calls recorded as abstentions

Project: network-visualiser **Found and fixed:** 17 September 2026 (Day 19) **Record written:** 21 September 2026
Status: closed. Two live instances of the same failure class, in other callers, were found while writing this record: D-45 and D-46. **Commits:** `git --no-pager log --oneline --grep="D-42"`

Summary

A classifier call that failed was stored as if the model had answered “I can’t classify this”. Nothing ever retried it, and nothing could tell it apart from a genuine answer.

The real network showed **183 of 442 people (41.4%) as needing review. The true figure was 128 (29.0%)**. 80 people carried a failed record; 55 of them were classifiable all along.

The failure was invisible for a structural reason. “Needs review” is the output the product is designed to show for hard titles, so extra grey nodes looked like the product working.

How it happened

Four links, each harmless alone:

1. **The token budget was shared with thinking.** `claude-sonnet-5` thinks adaptively, and `max_tokens` caps thinking and answer together. At `MAX_TOKENS = 2000`, a batch of 20 titles could spend the budget before the JSON answer was complete.
2. **The parser turned an unusable response into abstentions.** `_parse_response` filled every slot of a truncated or malformed batch with `{"soc": None, "confidence": "low", "why": "unparsed"}`. Its docstring gives the reasoning: a malformed response should degrade to “no claim” rather than a wrong claim. That reasoning is right about wrong claims. It missed that “no claim” is also a claim: that the model declined.
3. **The cache made a transient failure permanent.** `classify_claude` wrote the padded records to the cache like any other answer, so those titles were never sent again.
4. **Downstream, a padded record was an ordinary abstention.** Nothing read the `why` field. Every surface counted those people as needs-review, and every consistency check agreed with every other, because they were all correctly counting the wrong thing.

Why nothing caught it

The failure had the same shape as a result. The one distinguishing field, `why == "unparsed"`, was in the cache the whole time, and no check read it. The gates were designed to catch figures that disagree; D-42 produced one wrong figure that every surface agreed on. Gates C and D both passed against 183, and correctly: they tested consistency, not provenance.

The same root cause had already been found and fixed in another caller. On Day 15, D-16 showed that adaptive thinking could consume the chat assistant’s entire budget and return no text. It was fixed in `src/assistant.py`, which raised the budget to 8000 and stopped retrying on `stop_reason=max_tokens`. The classifier calls the same model with the same kind of budget, and it wasn’t checked. This is the same pattern as D-18 and D-32: a fix applied to the site where a failure showed up, not to every site that shared its cause.

The inflated figure acquired an explanation. The Day 17 Gate C addendum explained the change from 168/430 (39.1%) to 183/442 (41.4%) as newer titles “drifting toward less classifiable strings”. That explanation was plausible and never tested. A number that moves in the expected direction still needs a cause, and the D-42 fix withdraws that one.

How it was found

Not by any gate or test. It was found by reading code while preparing the P2.9a paid measurement run, and connecting three facts: D-16’s finding, the parser’s padding, and the cache write.

It was then treated as a hypothesis and tested at zero API cost: counting cached records with `why == "unparsed"` found **118 of 547 cached titles**. `tools/probe_unparsed.py` then replayed a batch of affected titles through the same model,

prompt and parser:

Call	stop_reason	Output tokens	Parsed
Batch of 20 at max_tokens=2000 (as shipped)	max_tokens	2000	0 / 20
Same 20 at max_tokens=8000	end_turn	1925	20 / 20
Batch of 5 at max_tokens=2000	end_turn	—	5 / 5

The shipped call hit the ceiling with the JSON cut off mid-array. The same titles parsed cleanly given room, or in a smaller batch.

Damage

	Before fix	After fix
Cached titles carrying a padded record	118 of 547	0
People carrying a padded record	80 of 442	0
Needs review	183 (41.4%)	128 (29.0%)
Classified	259	314
SOC groups present	—	19

The 118 poisoned entries were purged and re-classified. Of the 80 affected people, 55 received a classification and 25 were genuine abstentions.

Earlier figures are affected by an amount that can't be measured now. The cache doesn't timestamp its entries, so it isn't possible to say which padded records already existed at the 430-row stage. Every needs-review figure recorded before 17 September should be read as an upper bound. That includes 168/430 (39.1%) in Gate A, the methodology document and the decision log.

The fix

1. **MAX_TOKENS 2000 → 8000.** The comment records the measured 1925 tokens for a full batch, so the number isn't arbitrary.
2. **_classify_batch replaces the single call.** A response with `stop_reason == "max_tokens"`, or a batch the parser had to pad entirely, is a failed call. It splits the batch in half and retries each half. A partially padded batch retries only the missing records. If a single title still can't be answered alone, `ClassificationFailed` is raised. An abstention is never produced from a failure.
3. **Failures are never cached, and successes survive one.** The cache write moved into a `finally`, so a run that fails partway keeps everything that succeeded, and a rerun pays only for the failures.

`tests/test_classifier_d42.py` covers this with six tests: a truncated batch is split and answered; nothing marked unparsed reaches the cache; a padded record is retried rather than cached; an unanswerable title raises instead of abstaining; successes survive a later failure; cached titles are never re-requested.

`tools/measure_synthetic.py` also marks a run **INVALID**, and exits non-zero, if any measured title carries an unparsed record. A rate inflated by failures can't be reported as measured.

Where the old figures appear

Records are history and aren't rewritten; each gets a dated correction note instead.

Location	Figure	Disposition
<code>docs/gate_c_record.md</code>	259 / 183 at 442	Dated correction note (documentation batch)
<code>docs/gate_d_record.md</code>	259 / 183 / 0 at 442	Dated correction note (documentation batch)
<code>docs/day17_gate_c_addendum.md</code>	39.1% → 41.4%, with the title-drift explanation	Dated correction note withdrawing the explanation

docs/gate_a_record.md, docs/phase1_5_methodology.md, docs/project_state_decision_log.md, docs/project_status_refined_p2.md	168 / 430 (39.1%)	Upper bound, as above; dated note
README.md	"~40% of a typical network"	Rewritten at P2.9 (the README is replaced there)
docs/phase2_4_definitions_of_done.md	"~40% ... abstains"	Dated note
docs/p2_9a_record.md	v1 band 36-46% on 41.4%	Already records the D-42 supersession

Sweep of the other callers

The lesson from D-16 → D-42 is that a failure found in one caller of a shared dependency has to be checked in every caller. On 21 September, every `messages.create` call under `src/` was checked:

Caller	Budget	Finding
src/claude_classifier.py	8000	Fixed by D-42
src/assistant.py (chat)	8000	Fixed by D-16: <code>stop_reason=max_tokens</code> returns a truncation note, never retries
src/self_enrichment.py	8000 (the classifier's constant)	D-46. Still converts an unparsed response into <code>tier="abstain"</code> , the D-42 shape. Not cached, so it doesn't persist, but the user is shown "no classification" for what was a failure.
src/dashboard/summaries.py	400	D-45. The same model at a fifth of the budget that D-42 showed was too small for a batch. The call never checks <code>stop_reason</code> , and a thinking-only response gives empty text, which returns <code>None</code> : "no summary", indistinguishable from a missing key. Needs a measurement to confirm how often it happens.

Rules carried forward

- **A failed call is never an answer.** A truncated, empty or unparseable response raises or retries. It never becomes an abstention, an empty summary or a default, and it is never cached.
- **Fix the class, not the site.** When a failure is traced to a shared dependency (a model, a budget, a parser), sweep every caller before closing the defect.
- **Consistency is not correctness.** Surfaces agreeing proves they share a derivation, not that its input is sound. A check on provenance (here: "no cached record is a parser default") is a different check and needs its own test.
- **An explanation attached to a moving figure is a hypothesis.** Test it, or label it untested.
- **P2.9b must define partial-failure semantics explicitly.** When a build loses a batch, it either aborts or returns a network that states what is missing (see `docs/phase2_scope_revision.md`).